

Demo Walk-through and Verification Guide

Step-by-step instructions for evaluators
to reproduce every result

Author: Maryam Mueen
Intern ID: DHC-1151

1 How to Use This Document

This guide is written for the evaluator. It mirrors the screen-recording video step for step, so you can either follow along while watching, or reproduce every step locally to verify the results for yourself.

Each step shows three things:

- *Do* — the exact command or click to perform.
- *Result* — what you should observe.
- *Why* — the security concept being demonstrated.

Authorisation. Every action is performed against locally-hosted instances on `localhost:3000` running on the evaluator’s own machine. No third-party systems are touched.

2 One-Time Setup

Prerequisites: Node.js 18 or later (Node 22 LTS recommended). Install dependencies for both apps:

```
cd vulnerable-app
npm install

cd ../secure-app
cp .env.example .env      # then edit JWT_SECRET to a long random value
npm install
```

Seeded accounts.

Vulnerable build: admin / admin123 (plaintext, on purpose).

Secure build: admin / Admin#2026Strong (bcrypt-hashed).

3 Verify the Vulnerable Build

3.1 Step 1 — Boot the vulnerable app

Do

In PowerShell window #1:

```
cd vulnerable-app
npm start
```

Open Chrome and visit `http://localhost:3000`. Walk through the signup, login, and profile pages.

Result

The indigo / violet “UserHub” interface loads. All three pages render correctly. The app feels polished — which is the point: real products can ship with the same flaws.

Why

Establishes the baseline. The evaluator needs to confirm a working app exists before the attacks land.

3.2 Step 2 — Attack A: SQL Injection on /login

Do

On /login, enter:

- Username: admin' OR '1'='1
- Password: anything

Click Login.

Result

Browser redirects to /profile logged in as admin. The uid=1 cookie is set. The query becomes `SELECT * FROM users WHERE username = 'admin' OR '1'='1' AND password = 'anything'` — the OR is always true.

Why (V-01, Critical)

The login query is built by string concatenation, so user input becomes part of the SQL syntax itself. Parameterised queries (with placeholders) prevent this entirely.

3.3 Step 3 — Attack B: Stored XSS in bio

Do

On the profile page, paste the following into the bio textarea and click Save:

```
<script>alert('XSS by evaluator');</script>
```

Reload the page. Open Dev Tools console and type:

```
document.cookie
```

Result

The alert dialog “XSS by evaluator” pops on every page-load. The console returns "uid=1" as a plain string — proving the session cookie is reachable from JavaScript.

Why (V-02 High + V-06 High)

Two defects combine. Unescaped output lets attacker JS execute; missing `HttpOnly` means that JS can steal the session cookie. Either fix alone reduces the impact; both together kill the chain.

3.4 Step 4 — Attack C: Missing security headers

Do

Dev Tools → **Network** tab → reload the page → click the document request → **Headers** sub-tab. Look for:

- X-Powered-By: Express — framework leak.
- No Content-Security-Policy.
- No Strict-Transport-Security.
- No X-Frame-Options.

Or run from a shell:

```
curl -I http://localhost:3000/
```

Result

The `X-Powered-By` header is present; the four security headers above are all absent.

Why (V-04 Medium, V-05 Low)

HTTP security headers are defence-in-depth. A strict CSP would have neutralised the XSS payload from Step 3 even if escaping had been forgotten.

3.5 Step 5 — Attack D: Public /users JSON**Do**

Open a fresh incognito window (no auth cookie). Visit:

```
http://localhost:3000/users
```

Result

JSON response with every user including their email address. No auth check, no projection of safe columns.

Why (V-07 Medium)

Broken access control. Authentication is missing on a sensitive endpoint, and the response over-shares columns the page never needed.

4 Verify the Secure Build**4.1 Step 6 — Stop vulnerable, boot secure app****Do**

In PowerShell #1: press `Ctrl+C` to stop the vulnerable app. Then in PowerShell #2:

```
cd secure-app
npm start
```

Reload `http://localhost:3000`.

Result

The emerald / teal “UserHub” interface loads with the green “Hardened” badge in the navigation bar. Six feature cards appear on the landing page.

4.2 Step 7 — Re-test Attack A on the secure build**Do**

Same payload: `admin' OR '1'='1` as username, `anything` as password.

Result

The page reloads with the error message “Invalid credentials”. No cookie is set. The browser stays on `/login`. The attempt also appears in `secure-app/security.log` as a `warn` entry with the literal payload in the `username` field.

Why

The fix is a single line of code (`db.prepare('... WHERE username = ?').get(username)`). The input is bound as a string parameter, not parsed as SQL.

4.3 Step 8 — Legit login, then re-test Attack B

Do

Log in legitimately as admin / Admin#2026Strong. On the profile page paste the same XSS payload into bio and Save:

```
<script>alert('XSS');</script>
```

Reload. Open Dev Tools console and type `document.cookie`.

Result

The bio displays as literal text `<script>alert('XSS');</script>` — no alert fires. The console `document.cookie` returns an empty string because the JWT cookie is `HttpOnly` and not reachable from JS.

Why

Three defences in depth. The EJS `<%= %>` tag escapes the payload into HTML entities; even if escaping was forgotten, the CSP would block inline scripts; and even if a script did execute, the `HttpOnly` flag means it cannot read the cookie.

4.4 Step 9 — Re-test Attack C: headers

Do

Dev Tools → Network → Headers, or:

```
curl -I http://localhost:3000/
```

Result

```
Content-Security-Policy: default-src 'self'; script-src 'self'; ...
Strict-Transport-Security: max-age=15552000; includeSubDomains
X-Content-Type-Options: nosniff
X-Frame-Options: SAMEORIGIN
Referrer-Policy: no-referrer
(no X-Powered-By)
```

Why

`helmet()` adds the whole security-header bundle in one line. `scriptSrc 'self'` blocks both inline scripts and remote scripts, closing the XSS attack surface from multiple angles.

4.5 Step 10 — Re-test Attack D: /users

Do

Visit `/users` in the incognito window. Then visit it again in the logged-in window.

Result

Incognito: 302 Found → `/login`.

Logged in: JSON array of usernames only — no emails, no IDs.

Why

The `requireLogin` middleware guards the route, and the query projects only the columns the page needs.

4.6 Step 11 — Inspect the winston audit log

Do

Open `secure-app/security.log` in any text editor. Scroll to the bottom (the latest entries from your verification run).

Result

```
{
  "level": "info",
  "message": "Application started",
  "port": 3000,
  ...
}
{
  "level": "warn",
  "message": "Failed login",
  "username": "admin' OR '1'='1",
  "ip": "::1",
  ...
}
{
  "level": "info",
  "message": "User logged in",
  "userId": 1,
  "username": "admin",
  ...
}
{
  "level": "info",
  "message": "Profile updated",
  "userId": 1,
  ...
}
```

Why

Prevention is only half the story. Without an audit trail you cannot tell who tried what or investigate after the fact. The SQLi attempt is preserved verbatim in the warn log so a SOC analyst could spot the pattern later.

5 Visual Contrast — Vulnerable vs Secure

Attack	Vulnerable build	Secure build
SQL injection on login	passes (logged in as admin)	blocked (“Invalid credentials”)
Stored XSS in bio	passes (<code>alert()</code> fires)	blocked (rendered as escaped text)
Cookie reachable in JS	passes (<code>document.cookie</code> returns it)	blocked (empty due to <code>HttpOnly</code>)
Missing security headers	passes (X-Powered-By leak, no CSP)	blocked (full helmet bundle)
Anonymous <code>/users</code>	passes (dumps emails)	blocked (302 to <code>/login</code>)
Brute force possible	passes (unlimited attempts)	blocked (429 after 20 / 15 min)

Note for the evaluator. If any step above does not behave as documented, please verify that the `secure-app/.env` file exists with a non-default `JWT_SECRET`, and that both `npm install` runs completed without errors.